



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 6, 2026

Content in this Practice I

1 Divide and Conquer algorithms

- Divide and Conquer algorithms
- Merge Sort

2 Merge Sort

- Merge Sort - Divide
- Merge Sort - Representation example
- Merge Sort - Merge example
- Merge Sort - Merge structogram
- Merge Sort - Summary

Divide and Conquer algorithms

Divide and Conquer algorithms

- *A divide-and-conquer algorithm breaks down a large problem into smaller subproblems, which themselves may be broken down into even smaller subproblems, and so forth. The recursion bottoms out when it reaches a base case and the subproblem is small enough to solve directly without further recursing.*
- A divide-and-conquer algorithm:
 - **Divide** the problem into one or more subproblems that are smaller instances of the same problem.
 - **Conquer** the subproblems by solving them recursively.
 - **Combine** the subproblem solutions to form a solution to the original problem.

Merge Sort

- Main idea:

Imagine that we have two already sorted arrays. Is there an efficient way to create a new sorted array from them?

- Merge Sort:

Merge sort is a recursive algorithm that continuously divides the array into two parts until it can no longer be divided further, that is, until each part contains only a single element (a single-element array is always sorted). Then the sorted subarrays are merged into one sorted array.

Merge Sort

Merge Sort

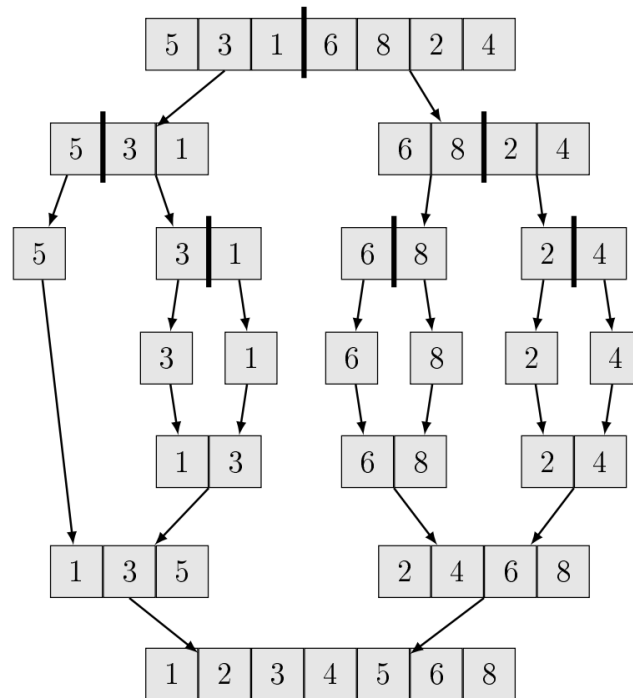
- Main idea:

Let's imagine we have two already sorted arrays. Is there a good way of creating a new combined sorted array from it?

- Merge Sort:

Merge sort is a recursive algorithm that continuously splits the array in half until it cannot be further divided i.e., the array has only one element left (an array with one element is always sorted). Then the sorted subarrays are merged into one sorted array.

Merge Sort - Representation



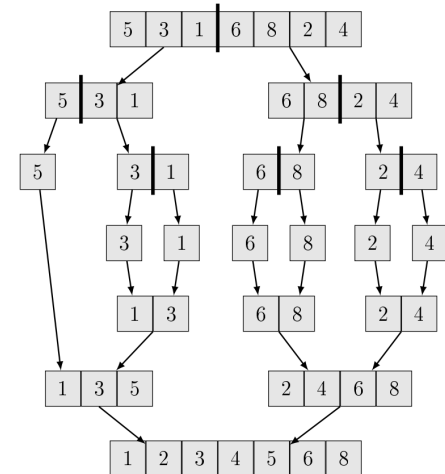
Merge Sort - Divide

$\text{mergeSort}(A : \mathcal{T}[n])$

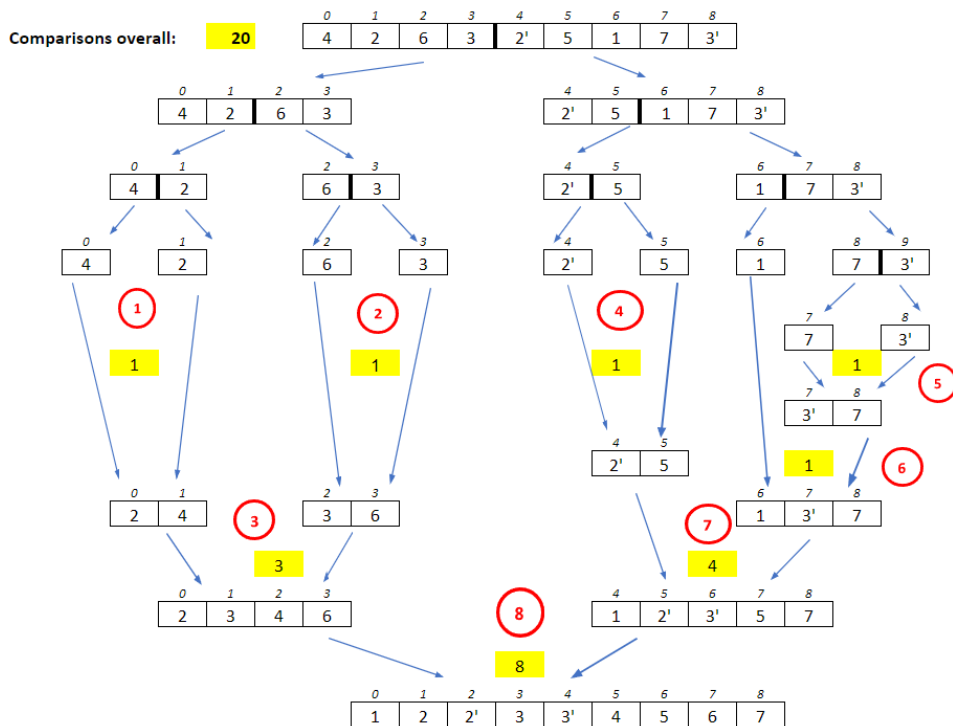
$B : \mathcal{T}[n] ; B[0..n] := A[0..n]$
// Sort $B[0..n]$ into $A[0..n]$ non-decreasingly:
$\text{ms}(B, A, 0, n)$

$\text{ms}(B, A : \mathcal{T}[] ; u, v : \mathbb{N})$

// Initially $B[u..v] = A[u..v]$.	
// Sort $B[u..v]$ into $A[u..v]$ non-decreasingly:	
$v - u > 1$	
$m := \lfloor \frac{u+v}{2} \rfloor$	SKIP
$\text{ms}(A, B, u, m)$ // Sort $A[u..m]$ into $B[u..m]$ non-decreasingly.	
$\text{ms}(A, B, m, v)$ // Sort $A[m..v]$ into $B[m..v]$ non-decreasingly.	
$\text{merge}(B, A, u, m, v)$ // merge $B[u..m]$ and $B[m..v]$ into $A[u..v]$	



Merge Sort - Representation example



Merge Sort - Representation example

1

		0	1	2	3	4	5	6	7	8
A		4	2	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

2

		0	1	2	3	4	5	6	7	8
A		2	4	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

3

		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

4

		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

merge(B,A,0,1,2) (from B to A)

		0	1	2	3	4	5	6	7	8
A		2	4	6	3	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

comparisons 1

merge(B,A,2,3,4) (from B to A)

		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		4	2	6	3	2'	5	1	7	3'

1

merge(A,B,0,2,4) (from A to B)

		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

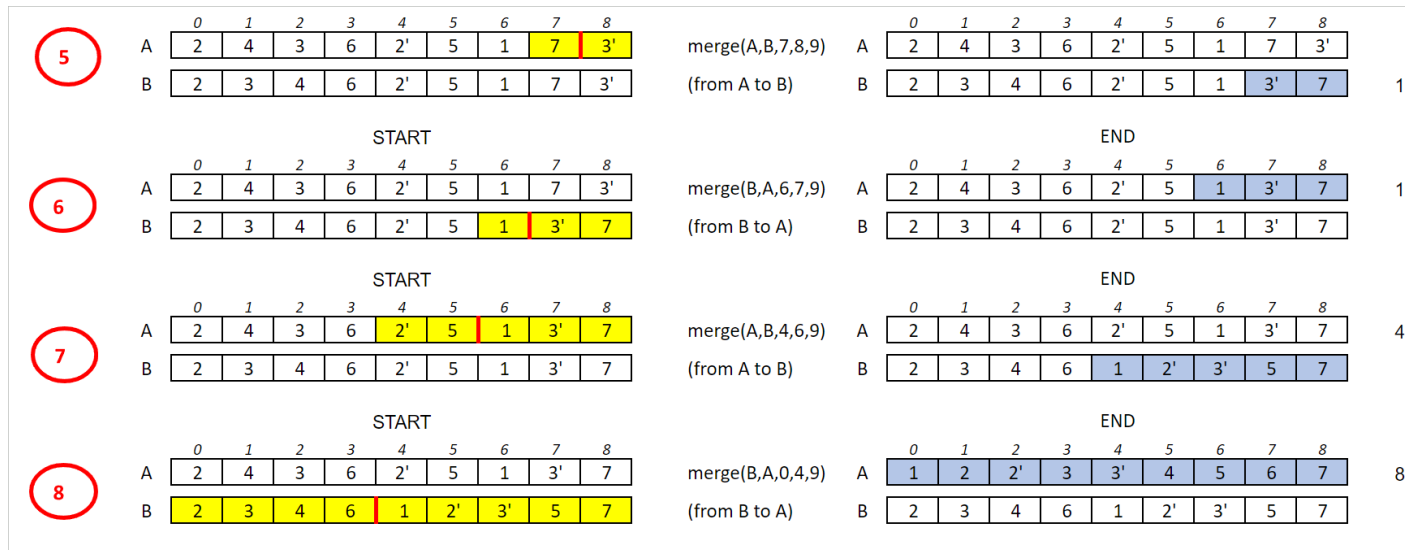
3

merge(B,A,4,5,6) (from B to A)

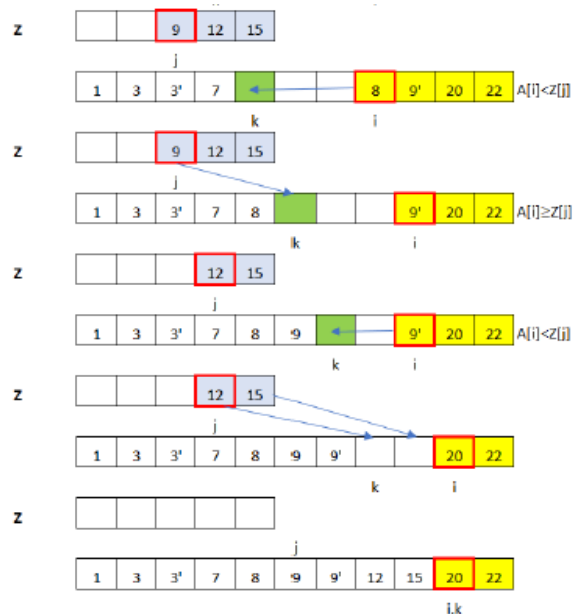
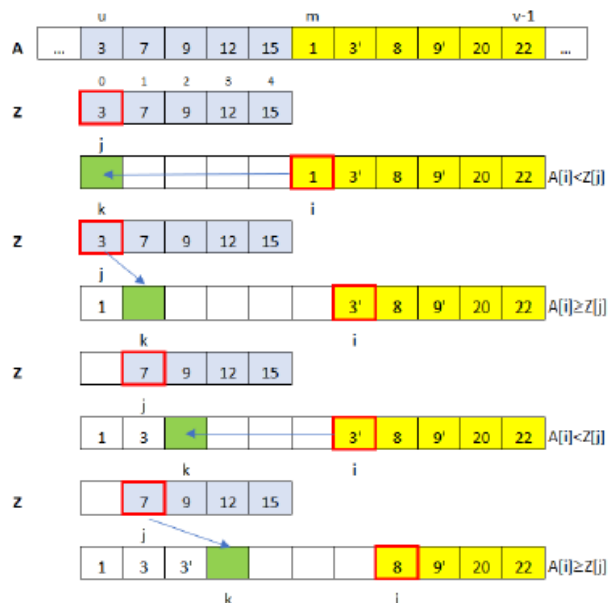
		0	1	2	3	4	5	6	7	8
A		2	4	3	6	2'	5	1	7	3'
B		2	3	4	6	2'	5	1	7	3'

1

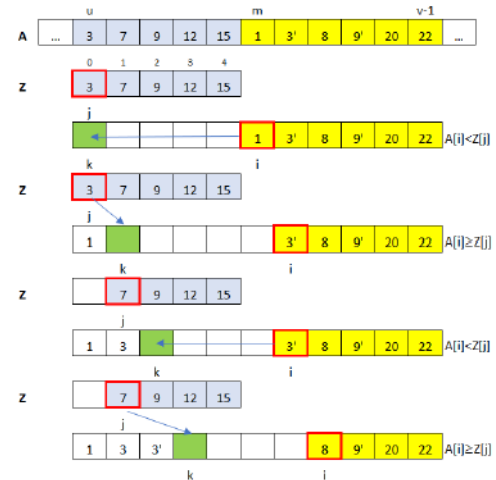
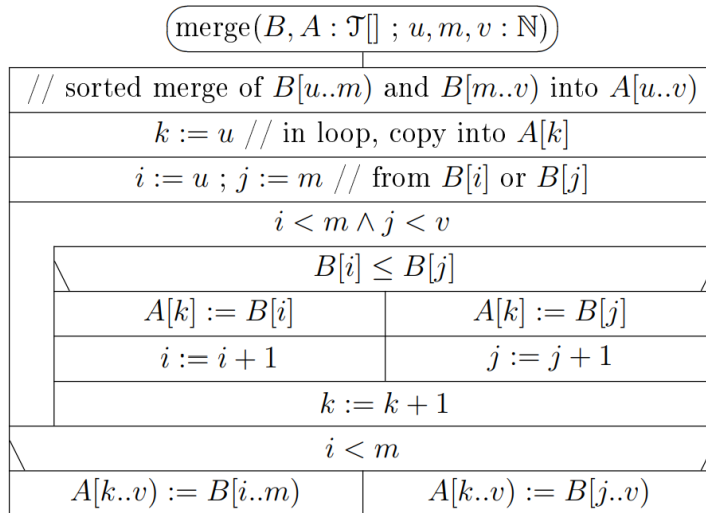
Merge Sort - Representation example



Merge Sort - Merge example



Merge Sort - Merge structogram



Merge Sort - Summary

Time complexity:

- $mT(n), AT(n), MT(n) \in O(n * \log(n))$

Space complexity

- $O(n)$
- We need to build a temporary array to merge the sorted arrays.

Stable: The sorting algorithms maintain the relative order of the duplicate data.

Thank you for your attention!